24 x 15 m 필드 Dual LiDAR 통합 운영 매뉴얼

명령어만 모은 현장용 문서: FIELD_COMMANDS_24x15.md

대상 장비: 지상 설치 Ouster OS1 - 128 두 대 캘리브레이션 타겟: 지름 30 cm, 높이 1.8 m 수직 원통 운용 모드: 1024x10 워크스페이스: ~/drone_ws

1. 먼저 이해할 것

캘리브레이션은 두 단계다.

1. 지면 평면으로 각 센서의 z / roll / pitch를 구한다.
2. 좌표를 아는 원통 타겟 9개로 x / y / yaw를 구한다.

z는 현재 2D x/y/yaw solver에 직접 들어가지 않는다. 반면 roll/pitch는 apply_tilt_correction:=true일 때 캡처한 3D 점을 먼저 기울기 보정한 후 XY를 사용하므로 x/y/yaw에 영향을 준다.

작은 각도에서 그 영향은 대략 다음과 같다.

```
delta_x ~= pitch * point_z
delta_y ~=  - roll * point_z
```

따라서 순서는 반드시 ground plane - > roll/pitch 입력 - > 9점 XY/yaw 보정이어야 한다.

2. 네트워크, PTP, 드라이버

```
ping - c 3 192.168.6.11
ping - c 3 192.168.6.12
```

안 되면:

```
nmcli con up lidar - network
ip - brief addr show enp12s0
```

PTP는 별도 터미널에서 계속 실행한다.

```
sudo ptp4l - f ~/drone_ws/ptp_gm.conf - m
```

드라이버를 실행한다.

```
ros2 launch dual_ouster_driver dual_os1.launch.py
```

토픽을 확인한다.

```
ros2 topic hz /lidar1/points
ros2 topic hz /lidar2/points
```

3. 방위각을 현장에서 혼자 결정하는 법

3.1 방위각 정의

ground_plane_node와 pointcloud_target_node는 원시 센서 좌표계에서 atan2(y,x)로 방위각을 계산한다.

```
          +y, +90 deg
            |
-x, 180 deg - - - - + - - - - +x, 0 deg
            |
          -y, -90 deg
```

범위의 의미:

azimuth 설정	선택 영역
min=-90, max=+90	센서 +x 방향 반원
min=+90, max=-90	+180/-180 deg를 통과하는 센서 -x 방향 반원
min=+100, max=-100	센서 -x 방향을 160도로 좁힌 영역

서로 반대편에 설치된 센서라도 센서 좌표축도 본체와 함께 회전한다. 따라서 양쪽 센서 모두 자기 좌표계의 -x가 필드를 향하도록 달았다면 둘 다 90 / -90을 사용한다. 이 매뉴얼의 설치 방향은 두 센서 모두 필드가 원시 -x 쪽에 놓이는 구성이므로 기본값으로 90 / -90을 사용한다.

3.2 axis_probe는 반드시 원시 좌표로 실행

현재 axis_probe의 기본 yaw_deg는 180도다. 기본값 그대로 보면 원시 포인트의 +x/-x가 뒤집혀 보일 수 있으므로 방향을 정할 때는 반드시 0으로 지정한다.

```
ros2 run dual_ouster_driver axis_probe --ros-args \
  -p input_topic:=/lidar1/points \
  -p yaw_deg:=0.0 \
  -p z_band:=1.2 \
  -p max_range:=6.0
```

LiDAR 2도 토픽만 바꿔 실행한다. 한 사람이 센서에서 3~5 m 떨어진 필드 중앙 방향에 서서 움직이고 가까워지는 sector를 확인한다.

- +x (0deg)가 반응: -90 / +90
- -x (180deg)가 반응: +90 / -90

처음부터 360도를 열지 않는다. 방향이 틀렸으면 반대 반원을 시험한다. 펜스나 차량이 포함되면 90/-90 -> 100/-100 -> 110/-110처럼 10도씩 좁힌다.

4. 지면 평면: z / roll / pitch

센서 앞쪽이 평지이고 장애물이 없는 조건에서는 range_max=10, 방위각 90/-90을 시작값으로 사용한다. 넓은 반원에서 얻은 지면점은 평면 법선의 roll/pitch 추정에 유리하다. inlier가 낮거나 다른 평면이 섞일 때만 100/-100, 110/-110처럼 10도씩 좁힌다.

```
# lidar1
ros2 run grid_sensor_calibration ground_plane_node --ros-args \
  -p input_topic:=/lidar1/points \
  -p sensor_label:=lidar1 \
  -p expected_height:=1.8 \
  -p num_frames:=20 \
  -p range_max:=10.0 \
  -p azimuth_min:=90.0 \
  -p azimuth_max:=-90.0
```

```
# lidar2
ros2 run grid_sensor_calibration ground_plane_node --ros-args \
  -p input_topic:=/lidar2/points \
  -p sensor_label:=lidar2 \
  -p expected_height:=1.7 \
  -p num_frames:=20 \
  -p range_max:=10.0 \
  -p azimuth_min:=90.0 \
  -p azimuth_max:=-90.0
```

다른 장소에서는 3장에서 실측한 반원을 넣는다. expected_height는 정답이 아니라 지면 crop의 중심이므로 줄자로 잰 대략적인 센서 높이를 넣는다.

판정 기준:

- inlier fraction은 가능하면 70% 이상
- residual sigma는 가능하면 20 mm 이하
- 20프레임의 z/roll/pitch spread가 작을 것
- range와 방위각을 조금 바꿔도 결과가 비슷할 것
- inlier가 낮고 sigma가 작은 경우 다른 평면이 섞였을 가능성이 있으므로 RViz로 확인

합격한 출력의 z/roll/pitch를 기록하고 8장의 XY/yaw 보정 명령에 그대로 입력한다.

5. 새 원통 타겟 설정

타겟은 지름 30 ㎝, 높이 1.8 ㎜다. 지름 30 cm이므로 원피팅 반지름은 그대로 0.15 m다. 높이가 늘었으므로 cluster extent는 2.2 m로 둔다.

아래 설정은 1.8 m 전체를 무조건 crop에 넣는 방식이 아니다. 지면 오검출을 피하려고 밑부분 약 30 cm를 버리고, 윗면 가장자리 영향도 피하려고 위쪽 약 10 cm를 버린 뒤 남은 수직 몸통으로 XY 원을 피팅한다. 따라서 1.8 m 타겟이 반영됐어도 z box의 폭은 약 1.4 m가 정상이다.

```
target_shape       = cylinder
target_radius      = 0.15
cluster_max_extent = 2.2
```

센서 높이를 h, 타겟 높이를 H=1.8이라 하면 지면을 제외하는 z crop 시작값은:

```
z_min ~= -h + 0.25~0.30
z_max ~= H - h - 0.10
```

센서 높이가 각각 1.770 m, 1.642 m일 때의 계산 예:

```
lidar1 h=1.770 -> z approx. [-1.47, -0.07]
lidar2 h=1.642 -> z approx. [-1.34, +0.06]
```

지면 경사와 센서 tilt 때문에 먼 지점에서는 raw sensor z가 달라질 수 있다. RViz에서 원통이 충분히 남고 지면은 빠지는지 확인한 뒤 범위에 약간의 여유를 준다.

원통은 반드시 수직으로 세우고 밑동 중심을 기준점 정중앙에 고정한다. 1.8 m 원통이 2도 기울면 꼭대기가 약 6.3 cm 이동한다. 이는 소프트웨어로 완전히 보정할 수 없다.

6. 원통 검출 노드

아래 명령은 필드가 원시 -x 방향이고 센서 높이가 각각 1.770 m, 1.642 m일 때의 시작값이다. 센서 높이가 달라지면 5장의 식으로 box의 z 범위를 다시 계산한다.

```
# lidar1
ros2 run grid_sensor_calibration pointcloud_target_node --ros-args \
  -p input_topic:=/lidar1/points \
  -p output_topic:=/sensor1/target_point \
  -p frame_id:=lidar1 \
  -p 'box:=[-30.0,30.0,-30.0,30.0,-1.47,-0.07]' \
  -p range_min:=1.0 -p range_max:=22.0 \
  -p azimuth_min:=90.0 -p azimuth_max:=-90.0 \
  -p cluster_tolerance:=0.20 \
  -p cluster_min_points:=5 \
  -p cluster_max_extent:=2.2 \
  -p selection:=largest \
  -p target_shape:=cylinder \
  -p target_radius:=0.15 \
  -p max_fit_residual:=0.04 \
  -p voxel_size:=0.0

# lidar2
ros2 run grid_sensor_calibration pointcloud_target_node --ros-args \
  -p input_topic:=/lidar2/points \
  -p output_topic:=/sensor2/target_point \
  -p frame_id:=lidar2 \
  -p 'box:=[-30.0,30.0,-30.0,30.0,-1.34,0.06]' \
  -p range_min:=1.0 -p range_max:=22.0 \
  -p azimuth_min:=90.0 -p azimuth_max:=-90.0 \
  -p cluster_tolerance:=0.20 \
  -p cluster_min_points:=5 \
  -p cluster_max_extent:=2.2 \
  -p selection:=largest \
  -p target_shape:=cylinder \
  -p target_radius:=0.15 \
  -p max_fit_residual:=0.04 \
  -p voxel_size:=0.0
```

검출 확인:

```
ros2 topic echo --once /sensor1/target_point
ros2 topic echo --once /sensor2/target_point
```

토픽이 나온다는 사실만으로 정확한 원통이라고 판단하면 안 된다. 다른 물체를 원통으로 선택한 상태에서도 토픽은 발행될 수 있으므로 다음 기준을 모두 확인한 뒤 캡처한다.

- 좌표가 수초 동안 수 cm 이내로 안정적
- fit residual이 가능하면 20 mm 이하
- 20~40 mm이면 원통 수직도와 주변 물체를 다시 확인
- 40 mm 초과면 현재 설정에서 기각됨
- 사람이 움직일 때 좌표가 따라가면 잘못된 후보
- RViz에서 실제 원통에 점이 모이는지 확인

7. 9개 기준점 선택

점은 중앙에 몰기보다 필드 전체에 넓게 퍼뜨려야 yaw와 필드 끝의 위치가 정확해진다. 구석점은 수학적으로 나쁜 점이 아니며 오히려 yaw에 유리하다. 다만 먼 센서에서도 검출이 안정적이고, 원통 중심을 정확히 경계 표식에 놓을 수 있어야 한다.

7.1 기본 배열

```
(0,0)   (12,0)   (24,0)
(0,6)   (12,6)   (24,6)
(0,15)  (12,15)  (24,15)
```

ROS 파라미터:

```
[0,0, 12,0, 24,0, 0,6, 12,6, 24,6, 0,15, 12,15, 24,15]
```

필드 가장자리까지 넓게 퍼진 배열은 yaw 추정과 필드 끝 위치 정확도에 유리하다. 가까운 (12,0), (12,15)에서는 원통 전체 높이가 보이지 않아도 수평 포인트가 충분하고 원피팅 residual이 안정적이면 사용할 수 있다. 가장 먼 구석은 센서에서 약 20.81 m다.

결정 전에 (0,0), (24,0), (0,15), (24,15) 네 점에서 양쪽 센서를 시험한다. 좌표가 안정되고 fit residual이 가능하면 20 mm 이하일 때 이 배열을 그대로 쓴다.

7.2 구석 검출이 불안할 때의 대체 배열

```
(3,0)   (12,0)   (21,0)
(3,6)   (12,6)   (21,6)
(3,15)  (12,15)  (21,15)
```

구석의 원거리 검출이 반복해서 실패할 때만 x를 한 칸 안쪽으로 옮긴다.

독립적인 절대 정확도 검증을 원하면 9개 fit 점 외에 2~3개 별도 검증점을 추가한다. 9점을 모두 fit에 사용해 얻은 RMS는 그 9점에 대한 내부 일관성이다.

8. XY / yaw 보정 노드

아래 명령은 기본 9점을 사용한다. 명령의 sensor_z/roll/pitch는 4장에서 합격 판정을 받고 기록한 값으로 바꿔 넣는다. nominal_x/y는 계산값을 강제하지 않고 설치오차 리포트에만 쓰지만, 줄자로 잰 실제 설치 위치를 넣는 것이 좋다.

```
# sensor1
ros2 run grid_sensor_calibration sensor_calibration_node  - -ros - args \
  - r __node:=sensor1_calibration \
  - p sensor_name:=sensor1  - p child_frame:=lidar1 \
  - p input_topic:=/sensor1/target_point \
  - p input_message_type:=point_stamped \
  - p capture_sample_count:=10 \
  - p auto_calibrate:=false \
  - p 'grid_points:=[0.0,0.0,12.0,0.0,24.0,0.0,0.0,6.0,12.0,6.0,24.0,6.0,0.0,15.0,12.0,15.0,24.0,15.0]' \
  - p calibration_file:=$HOME/grid_calibration/field_24x15/sensors_calibration.yaml \
  - p nominal_x:=12.0  - p nominal_y:= - 2.0  - p nominal_yaw:=0.0 \
  - p apply_tilt_correction:=true \
  - p sensor_z:=1.770  - p sensor_roll:= - 0.025492  - p sensor_pitch:= - 0.000386

# sensor2
ros2 run grid_sensor_calibration sensor_calibration_node  - -ros - args \
  - r __node:=sensor2_calibration \
  - p sensor_name:=sensor2  - p child_frame:=lidar2 \
  - p input_topic:=/sensor2/target_point \
  - p input_message_type:=point_stamped \
  - p capture_sample_count:=10 \
  - p auto_calibrate:=false \
  - p 'grid_points:=[0.0,0.0,12.0,0.0,24.0,0.0,0.0,6.0,12.0,6.0,24.0,6.0,0.0,15.0,12.0,15.0,24.0,15.0]' \
  - p calibration_file:=$HOME/grid_calibration/field_24x15/sensors_calibration.yaml \
  - p nominal_x:=12.0  - p nominal_y:=17.0  - p nominal_yaw:=0.0 \
  - p apply_tilt_correction:=true \
  - p sensor_z:=1.642  - p sensor_roll:=0.011357  - p sensor_pitch:=0.002479
```

센서 위치나 설치 각도가 바뀌면 반드시 4장에서 새로 구한 z/roll/pitch와 실측 nominal 위치를 넣는다.

9. 캡처

반드시 grid_points 순서대로 이동한다.

```
(0,0) -> (12,0) -> (24,0)
  -> (0,6) -> (12,6) -> (24,6)
  -> (0,15) -> (12,15) -> (24,15)
```

9.1 트리거 전 예상 좌표표

다음 명목 설치를 가정한 raw sensor-frame XY다.

```
LiDAR 1: world=(12, -2, 2), ideal_yaw=-90 deg, sensor -x faces north
LiDAR 2: world=(12, 17, 2), ideal_yaw=+90 deg, sensor -x faces south
roll=pitch=0 approximation
```

ros2 topic echo --once /sensorN/target_point의 point.x, point.y를 아래 값과 비교한다. r은 XY 거리, az는 atan2(y,x) 방위각이다.

기준점	LiDAR 1 예상 (x, y) m	r / az	LiDAR 2 예상 (x, y) m	r / az
(0,0)	(-2.00, -12.00)	12.17 m / -99.5 deg	(-17.00, +12.00)	20.81 m / +144.8 deg
(12,0)	(-2.00, 0.00)	2.00 m / 180.0 deg	(-17.00, 0.00)	17.00 m / 180.0 deg
(24,0)	(-2.00, +12.00)	12.17 m / +99.5 deg	(-17.00, -12.00)	20.81 m / -144.8 deg
(0,6)	(-8.00, -12.00)	14.42 m / -123.7 deg	(-11.00, +12.00)	16.28 m / +132.5 deg
(12,6)	(-8.00, 0.00)	8.00 m / 180.0 deg	(-11.00, 0.00)	11.00 m / 180.0 deg
(24,6)	(-8.00, +12.00)	14.42 m / +123.7 deg	(-11.00, -12.00)	16.28 m / -132.5 deg
(0,15)	(-17.00, -12.00)	20.81 m / -144.8 deg	(-2.00, +12.00)	12.17 m / +99.5 deg
(12,15)	(-17.00, 0.00)	17.00 m / 180.0 deg	(-2.00, 0.00)	2.00 m / 180.0 deg
(24,15)	(-17.00, +12.00)	20.81 m / +144.8 deg	(-2.00, -12.00)	12.17 m / -99.5 deg

이 표는 원통을 잘못 잡았는지 찾기 위한 명목값이지 10 cm 판정 기준은 아니다. 캘리브레이션 전에는 실제 설치 x/y 오차와 yaw 오차를 아직 모르기 때문이다. yaw가 4도만 달라도 19 m 지점의 예상 XY가 약 1.3 m 달라진다. 따라서 시작 전에는 표에서 약 1~1.5 m 이내이며 부호 · 거리 · 방향이 맞는지 확인하고, 수초 동안 좌표가 안정적인지를 더 중요하게 본다. 수 m 이상 다르거나 다른 행의 좌표로 순간이동하면 캡처하지 않는다.

point.z는 원통 밑동 좌표가 아니라 crop에 남은 원통 점들의 z 중앙값이다. 센서가 2 m 높이이고 z box가 [-1.70, -0.30]이면 출력 z도 그 범위 안에 있어야 하지만, 정확히 -2.0이나 특정 고정값이 나오는 것은 아니다. 트리거 전 비교의 핵심은 x/y, range, azimuth와 시간 안정성이다.

각 점에서:

1. 원통 밑동 중심을 그리드점 정중앙에 놓는다.
2. 수평계로 수직을 맞추고 바람에 움직이지 않게 고정한다.
3. 사람은 원통과 검출 영역에서 빠진다.
4. 양쪽 target topic의 좌표와 residual을 확인한다.
5. 두 센서를 캡처한다.

```
ros2 service call /sensor1_calibration/capture_point std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/capture_point std_srvs/srv/Trigger
```

응답의 grid 좌표가 현재 위치와 일치하는지 확인한다.

```
ros2 service call /sensor1_calibration/status std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/status std_srvs/srv/Trigger
```

재캡처와 복구:

```
# recapture index 4
ros2 param set /sensor1_calibration capture_index 4
ros2 service call /sensor1_calibration/capture_point std_srvs/srv/Trigger

# undo last capture
ros2 service call /sensor1_calibration/undo_point std_srvs/srv/Trigger

# reset all captures
ros2 service call /sensor1_calibration/reset std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/reset std_srvs/srv/Trigger
```

10. 계산과 정확도 해석

```
ros2 service call /sensor1_calibration/run_calibration std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/run_calibration std_srvs/srv/Trigger
```

출력된 RMS, 평균 거리 오차, 최대 거리 오차와 각 기준점 residual을 기록한다. 권장 목표는 RMS 2~5 cm다. RMS가 5~10 cm이면 residual이 큰 점을 확인하고 재캡처하며, 10 cm를 넘으면 캡처 순서, 원통 중심, 수직도와 잘못된 후보 검출부터 다시 확인한다. 최대 오차는 시스템 허용 오차보다 충분히 작아야 한다.

RMS는 9개 기준점에서 얻은 내부 fitting residual이며 드론 최종 추적 오차가 아니다. 다음 오차는 별도로 더해질 수 있다.

- 그리드점 측량 및 원통 밑동 배치 오차
- 원통 수직도와 중심 피팅 오차
- 드론 클러스터 중심과 실제 기준점의 차이
- 두 센서 시간차와 융합 오차

특정 기준점의 residual만 크면 전체를 다시 하지 말고 해당 인덱스를 재캡처한다.

추가 판정:

- 두 센서 yaw 차이가 약 180도인지 확인
- 계산된 x/y가 줄자로 잰 센서 위치와 맞는지 확인
- 별도 검증점에서 변환 후 좌표 오차를 측정

11. LLM 없이 문제를 찾는 순서

A. ground plane이 이상할 때

1. axis_probe ... -p yaw_deg:=0.0으로 원시 필드 방향을 다시 확인한다.
2. 360도를 열지 말고 -90/90과 90/-90 두 반원을 각각 시험한다.
3. inlier가 높은 쪽을 고른다.
4. 다른 평면이 섞이면 10도씩 좁힌다.

5. range_max를 20 - > 15 - > 10 m로 줄여 결과 안정성을 비교한다.
6. inlier, sigma, 프레임 spread 세 가지가 모두 좋은 결과만 채택한다.

B. 원통 토픽이 안 나올 때

```
ros2 run rviz2 rviz2
```

Fixed Frame을 해당 LiDAR로 두고 PointCloud2를 표시한 뒤 다음 순서로 확인한다.

1. raw cloud에 원통이 실제로 보이는가
2. azimuth 반원이 맞는가
3. z box가 원통을 포함하고 지면을 제외하는가
4. range 범위가 맞는가
5. cluster_min_points:=5로 충분한가
6. 로그의 circle fit residual이 40 mm 이하인가

원피팅만 문제인지 확인할 때만 임시로 다음을 사용한다.

```
- p target_shape:=centroid
```

centroid로는 잡히는데 cylinder로 안 잡히면 실제 지름, 수직도, residual을 확인한다. centroid 결과는 보이는 앞면 쪽으로 약 0.7 * radius, 즉 약 10.5 cm 치우칠 수 있으므로 최종 캘리브레이션에는 사용하지 않는다.

C. 잘못된 물체가 잡힐 때

- 사람을 검출 영역에서 내보낸다.
- 방위각과 range를 현재 그리드점 주변으로 좁힌다.
- largest가 사람이나 구조물을 선택할 수 있음을 기억한다.
- 좌표가 순간이동하거나 residual이 20 mm를 계속 넘으면 캡처하지 않는다.
- RViz에서 후보가 실제 원통인지 확인한다.

12. 반영과 검증

최종 12개 값을 다음 파일에 반영한다.

```
src/drone_localization/params/lidar_localization_os1.yaml
```

```
lidar1_x: ...
lidar1_y: ...
lidar1_z: ...
lidar1_roll: ...
lidar1_pitch: ...
lidar1_yaw: ...

lidar2_x: ...
lidar2_y: ...
lidar2_z: ...
lidar2_roll: ...
lidar2_pitch: ...
lidar2_yaw: ...
```

YAML만 수정하면 재빌드는 필요 없다. 트래커를 재시작하고 확인한다.

```
ros2 topic echo /drone/estimated_pose
```

가능하면 좌표를 아는 별도 위치에 정지 물체 또는 드론을 놓고 실제 위치와 /drone/estimated_pose를 비교한다. 이것이 캘리브레이션 RMS와 별개인 최종 시스템 정확도 검증이다.

13. 보정 직후 두 점군 겹침 확인

숫자 RMS만 확인하고 바로 운영하지 않는다. 12절에서 새 x/y/z/roll/pitch/yaw를 lidar_localization_os1.yaml에 반영한 다음, 두 센서의 전체 점군을 각각 world 좌표로 변환해 같은 RViz 화면에서 겹쳐 본다. 이 확인에는 원통 검출 노드나 캘리브레이션 노드를 다시 실행할 필요가 없다.

먼저 1024x10 드라이버 하나만 실행 중인지 확인한다. 이미 실행 중이면 중복 실행하지 않는다.

```
source ~/drone_ws/install/setup.bash
ros2 launch dual_ouster_driver dual_os1.launch.py
```

새 터미널에서 최신 localization YAML을 사용해 트래커를 실행한다. 드라이버는 앞 터미널에서 이미 실행 중이므로 반드시 start_driver:=false를 준다.

```
source ~/drone_ws/install/setup.bash
ros2 launch drone_localization lidar_localization.launch.py \
  start_driver:=false
```

세 번째 터미널에서 겹침 확인 전용 RViz 설정을 연다.

```
source ~/drone_ws/install/setup.bash
rviz2 -d "$(ros2 pkg prefix dual_ouster_driver)/share/dual_ouster_driver/rviz/calibration_overlap.rviz"
```

RViz의 Fixed Frame은 world이며 다음 두 토픽을 서로 다른 색으로 표시한다.

```
LiDAR 1: /lidar1/points_world
LiDAR 2: /lidar2/points_world
```

13.1 무엇을 확인할 것인가

두 센서가 모두 볼 수 있는 필드 중앙의 고정 물체를 기준으로 본다. 가능하면 1.8 m 원통을 (12,6), (6,7.5), (18,7.5)처럼 서로 떨어진 3곳에 차례로 세워 확인한다. 한 점에서만 겹치는 것은 translation 오차가 rotation 오차를 우연히 상쇄한 결과일 수 있으므로 중앙 한 점만 보고 통과시키지 않는다.

- 평평한 지면이 양쪽 색에서 같은 z=0 부근에 놓이는가
- 같은 원통의 수직 몸통과 중심이 두 색에서 겹치는가
- 필드 중앙뿐 아니라 좌우로 떨어진 위치에서도 같은 방향으로 겹치는가
- 고정 구조물의 모서리와 필드 경계가 같은 world 좌표에 놓이는가
- 정지 물체는 겹치지만 움직이는 물체만 둘로 갈라지는가

센서가 서로 반대편에서 물체를 보기 때문에 물체의 앞면과 뒷면, 가려진 면은 완전히 같은 모양이 아닐 수 있다. 표면 전체의 외곽이 아니라 같은 기둥의 중심선, 지면 높이, 고정 모서리처럼 양쪽에서 공통으로 관측되는 특징을 비교한다.

13.2 어긋나는 모양으로 원인 구분

RViz에서 보이는 현상	우선 확인할 보정값 또는 원인
필드 전체에서 비슷한 방향과 크기로 평행 이동	x/y translation
중앙은 맞지만 멀어질수록 수평 간격 증가	yaw
두 지면 높이가 일정하게 위아래로 분리	z
한쪽 지면이 거리에 따라 위/아래로 벌어짐	roll/pitch 또는 잘못 선택한 ground plane
고정 물체는 겹치고 움직이는 물체만 분리	PTP/scan phase와 시간차, 캘리브레이션 문제 아님
한 위치만 맞고 다른 위치에서 반대 방향으로 벌어짐	yaw와 x/y가 함께 틀렸거나 9점 대응 오류

겹침이 나쁘면 운영으로 넘어가지 말고 다음 순서로 돌아간다.

1. YAML에 반영한 12개 값과 센서 번호가 바뀌지 않았는지 확인한다.
2. 지면이 틀리면 ground plane의 z/roll/pitch부터 다시 확인한다.
3. 수평으로 벌어지면 문제 기준점을 재캡처해 x/y/yaw를 다시 계산한다.
4. 고정 물체에서 양쪽 색이 안정적으로 겹친 뒤에만 운영 절차로 넘어간다.

이 RViz 확인은 전체 world cloud를 발행하므로 성능 측정용 설정이 아니다. 보정 확인이 끝나면 아래 절차로 관련 프로세스를 종료하고 운영 구성을 새로 시작한다.

14. 캘리브레이션 종료와 운영용 4개 터미널

14.1 남아 있는 캘리브레이션 프로세스 종료

가능하면 각 launch 터미널에서 Ctrl+C로 먼저 정상 종료한다. 어느 터미널에서 실행했는지 불분명하거나 프로세스가 남았을 때만 아래 pkill을 사용한다. pkill python3, pkill ros2처럼 범위가 넓은 명령은 다른 프로그램까지 종료하므로 사용하지 않는다.

```
# calibration / target extraction / TF helpers
pkill -INT -f "ground_plane_node|pointcloud_target_node|sensor_calibration_node"
pkill -INT -f "sensor_tf_publisher|point_transform_node|calibration_viz_node"
pkill -INT -f "[s]tatic_transform_publisher"

# tracker, GUI, scorer, visualization
pkill -INT -f "lidar_localization.launch.py|lidar_drone_tracker"
pkill -INT -f "competition_gui|compet_score|estimate_viz"
pkill -INT -x rviz2

# LiDAR drivers and PTP
pkill -INT -f "dual_os1.launch.py"
pkill -INT -x os_driver
sudo pkill -INT -x ptp4l
```

2~3초 뒤 남은 ROS 노드를 확인한다.

```
ros2 node list
```

위 명령은 ROS daemon 자체를 종료하지 않으므로 종료 직후 사라진 노드가 잠깐 보일 수 있다. 몇 초 뒤 다시 확인한다. os_driver_lidar1, os_driver_lidar2, 캘리브레이션 노드, tracker가 남아 있지 않으면 운영 구성을 시작한다.

14.2 운영용 터미널 1: PTP

이 터미널은 운영이 끝날 때까지 닫지 않는다.

```
cd ~/drone_ws
sudo ptp4l -f ~/drone_ws/ptp_gm.conf -m
```

에러 반복 없이 master 상태가 유지되는지 확인한다.

14.3 운영용 터미널 2: 두 LiDAR 드라이버

```
source ~/drone_ws/install/setup.bash
ros2 launch dual_ouster_driver dual_os1.launch.py
```

두 센서가 모두 active가 된 뒤 다음을 한 번씩 확인한다.

```
ros2 topic hz /lidar1/points
ros2 topic hz /lidar2/points
```

두 토픽 모두 약 10 Hz여야 한다. 확인이 끝나면 topic hz만 Ctrl+C로 끝내고 드라이버 launch는 계속 유지한다.

14.4 운영용 터미널 3: 대회 GUI

GUI는 START RUN을 눌렀을 때 tracker와 scorer를 시작한다. LiDAR 드라이버는 터미널 2에서 계속 실행한다.

```
source ~/drone_ws/install/setup.bash
ros2 run competition_gui competition_gui
```

START RUN을 누른 뒤 GUI 로그에서 다음을 확인한다.

- tracker와 scorer가 ready 상태인가
- 출력 폴더의 team, ID, run 번호가 맞는가
- update=10.0 Hz, fusion_mode=position으로 시작했는가
- 두 센서의 위치와 RPY가 방금 반영한 값인가

14.5 운영용 터미널 4: 가벼운 위치 시각화

운영 중에는 전체 /lidarN/points_world를 그리는 RViz 대신 위치, cluster, 공분산만 표시하는 object 전용 설정을 사용한다. 아래 두 명령은 같은 터미널에서 실행된다. estimate_viz는 background로 실행되고 RViz가 foreground로 열린다.

```
source ~/drone_ws/install/setup.bash
ros2 run drone_localization estimate_viz &
ESTIMATE_VIZ_PID=$!
rviz2 -d "$(ros2 pkg prefix dual_ouster_driver)/share/dual_ouster_driver/rviz/tracker_view_object.rviz"
kill -INT "$ESTIMATE_VIZ_PID" 2>/dev/null
```

RViz Fixed Frame은 world다. START RUN 이후 다음을 확인한다.

- /drone/estimated_pose가 실제 드론을 따라가는가
- EstimateText 좌표가 필드 x=0~24, y=0~15 범위와 맞는가
- 공분산 표시가 갑자기 커지거나 위치가 오래 coast하지 않는가
- 양쪽 센서 후보가 반복해서 크게 갈라지지 않는가

처리 주기 디버깅을 할 때는 GUI와 이 시각화 터미널을 종료하고 15장의 단독 트래커 명령을 사용한다. 그래야 GUI가 두 번째 트래커를 띄우거나 RViz 부하가 측정에 섞이지 않는다.

15. 야외 처리 주기 디버깅

처리 주기가 설정값인 10 Hz보다 낮거나 CSV 간격이 불규칙할 때 이 절차를 수행한다. raw_detections.csv는 검출 성공 때만 기록하고 drone_trajectory.csv는 검출 또는 coast 때 기록하므로, 어느 CSV도 LiDAR 원본 수신율 그 자체는 아니다. 아래 순서로 입력, 트래커 처리, 검출 출력을 분리해서 확인한다.

15.1 시험 전에 지킬 것

- LiDAR 드라이버는 센서당 하나만 실행한다. GUI와 수동 launch가 동시에 드라이버를

시작하면 같은 UDP 스트림을 두 프로세스가 경쟁할 수 있다.

- 직접 트래커를 실행할 때는 GUI가 실행한 트래커를 먼저 종료한다. 트래커도 하나만 둔다.
- 모든 터미널에서 source ~/drone_ws/install/setup.bash를 실행한다.
- PTP를 켜고 두 센서의 네트워크 연결이 안정된 뒤 측정한다.
- 각 시험은 최소 20~30초 유지하고 로그를 저장한다.

네트워크 오류 카운터도 먼저 확인한다.

```
ip -s link show enp12s0
```

RX errors, dropped가 시험 중 계속 증가하면 트래커보다 네트워크와 드라이버를 먼저 확인한다.

15.2 1단계: 트래커 없이 LiDAR 입력 확인

두 LiDAR만 켠 상태에서 서로 다른 터미널로 동시에 실행한다.

```
ros2 topic hz /lidar1/points
ros2 topic hz /lidar2/points
```

20~30초 동안 확인할 값은 average rate, min, max, std dev다. 두 토픽 모두 대략 9.5~10.5 Hz이고 주기가 약 0.1초로 안정적이어야 한다. 한 토픽만 느리면 해당 센서의 드라이버, 케이블, NIC, 센서 모드를 확인한다. 둘 다 2.5~3 Hz라면 트래커가 아니라 센서 또는 드라이버 단계의 문제다.

메시지 크기와 네트워크 부하도 기록한다.

```
ros2 topic bw /lidar1/points
ros2 topic bw /lidar2/points
```

QoS 확인:

```
ros2 topic info -v /lidar1/points
ros2 topic info -v /lidar2/points
```

현재 트래커 입력 subscription은 BEST_EFFORT, KEEP_LAST, depth 5다. 고속 센서에는 일반적인 설정이며 이것이 출력 주기를 3 Hz로 제한하지는 않는다. 처리 지연이 생기면 오래된 프레임을 쌓지 않고 버리게 하므로 프레임 누락이 결과로 나타날 수 있다. 원인 확인 전 RELIABLE로 바꾸면 backlog와 지연이 커질 수 있으므로 먼저 변경하지 않는다.

15.3 2단계: 현재 설정 그대로 병목 시간 측정

드라이버는 계속 실행하고 GUI와 기존 트래커는 종료한 뒤 다음 트래커 하나만 실행한다.

```
ros2 run drone_localization lidar_drone_tracker  - - ros - args \
  - - params - file /home/uscl2026/drone_ws/src/drone_localization/params/lidar_localization_os1.yaml \
  - p debug_timing:=true \
  - p debug_timing_period:=2.0 \
  2>&1 | tee "$HOME/competition_results/debug_timing_current.log"
```

로그의 의미:

항목	의미	10 Hz일 때 기대값
store frames	센서 subscription이 실제 받은 프레임	센서별 약 20/2초
pc2	PointCloud2를 NumPy로 변환한 평균 시간	작을수록 좋음
tf	전체 점을 world 좌표로 변환한 평균 시간	작을수록 좋음
process ... processed	페어 선택 후 처리한 프레임 수	약 18~20/2초
process avg	선택된 프레임 한 회의 전체 처리 시간	100 ms 미만, 가능하면 70 ms 이하
no_frame_ticks	타이머가 돌았지만 새 프레임이 없던 횟수	안정 입력에서는 거의 0
world_pub	전체 world cloud 재생성 · 발행 시간	크면 시각화 출력이 병목
roi_crop	필드/고도 ROI crop 시간	보통 작아야 함
voxel, sor	다운샘플 및 이상점 제거 시간	합계가 100 ms 예산을 넘지 않아야 함
filtered_pub	필터 cloud 재생성 · 발행 시간	크면 시각화 출력이 병목
dbscan, select	군집화와 드론 후보 선택 시간	ROI 점 수가 많을수록 증가

판정 방법:

- 원본 topic hz는 10 Hz인데 store frames가 센서별 5~6/2초이면 subscription

콜백이 밀리거나 전체 executor가 포화된 것이다.

- store frames는 약 20인데 processed만 5~6이면 타이머 처리, 프레임 선택 또는

검출 파이프라인이 밀리는 것이다.

- process avg가 100 ms를 넘으면 설정한 10 Hz를 계산상 유지할 수 없다.
- PTP 로그의 timestamp delta가 80 ms 이하여도 처리 속도가 보장되는 것은 아니다.

이 값은 두 센서의 시간 정렬 상태를 판단하는 값이다.

15.4 3단계: 시각화용 cloud 출력을 끄고 비교

현재 코드의 기본값은 publish_world_clouds:=true, publish_filtered_clouds:=true다. LiDAR 한 프레임이 약 12만 포인트이므로 두 센서의 전체 cloud를 매번 다시 만드는 비용이 가장 먼저 의심된다. RViz를 사용하지 않아도 메시지 생성 비용은 발생한다.

현재 설정 시험을 종료한 뒤 다음 명령으로 같은 장소에서 다시 20~30초 측정한다.

```
ros2 run drone_localization lidar_drone_tracker  - - ros - args \
  - - params - file /home/uscl2026/drone_ws/src/drone_localization/params/lidar_localization_os1.yaml \
  - p debug_timing:=true \
  - p debug_timing_period:=2.0 \
  - p publish_world_clouds:=false \
  - p publish_filtered_clouds:=false \
  2>&1 | tee "$HOME/competition_results/debug_timing_no_cloud_pub.log"
```

- processed가 5~6에서 18~20/2초로 증가하면 전체/필터 cloud 발행이 주원인이다.
- world_pub + filtered_pub 시간은 작지만 여전히 느리면 pc2, tf, sor,

dbscan 중 가장 큰 항목을 본다.

- 이 비교에서 개선이 확인된 뒤에만 대회용 YAML에 두 publish 값을 false로 고정한다.

위치 결과 /drone/estimated_pose와 CSV 기록에는 이 두 시각화 토픽이 필요하지 않다.

15.5 빈 필드 시험과 물체 시험을 분리

두 LiDAR만 켠 빈 필드 시험으로도 PointCloud2 변환, 전체 좌표변환, world cloud 발행 병목은 확인할 수 있다. 그러나 ROI 안에 점이 거의 없으면 코드는 SOR와 DBSCAN 전에 일찍 종료하므로 실제 드론 검출 부하는 측정되지 않는다.

따라서 cloud 발행 비교 후 다음 두 상태를 각각 20~30초 측정한다.

1. ROI가 비어 있는 평지
2. 실제 드론 또는 비슷한 크기의 물체가 z=0.8~3.5 m ROI 안에 있는 상태

두 번째 상태에서만 roi, filtered 평균 포인트 수와 sor, dbscan, select 시간을 최종 판단한다. 사람이 ROI 안에 많이 들어오면 부하와 오검출 조건이 달라지므로 최종 시험 때는 불필요한 인원을 빼는 것이 좋다.

드론이 실제로 검출되는 상태에서는 최종 출력률도 별도 터미널에서 확인한다.

```
ros2 topic hz /drone/estimated_pose
```

목표는 입력과 처리 모두 약 10 Hz이며, 출력은 드론이 계속 정상 검출되는 구간에서 이에 가까워야 한다. 출력만 낮고 processed는 10 Hz이면 성능 문제가 아니라 검출 실패와 coast/reset 조건을 확인한다.

15.6 현장 판정표

관측 결과	우선 판단	다음 조치
원본 두 토픽 모두 약 10 Hz	센서/드라이버 정상	트래커 timing 비교 진행
원본 한 토픽만 느림	해당 센서, NIC, 케이블 또는 드라이버 문제	드라이버 로그와 RX 오류 확인
원본은 10 Hz, store가 낮음	단일 executor 포화 또는 subscription drop	cloud publish OFF 비교
world_pub가 큼	전체 cloud 시각화가 병목	대회 운용에서 world cloud OFF
filtered_pub가 큼	필터 cloud 시각화가 병목	대회 운용에서 filtered cloud OFF
sor가 큼	ROI 포인트가 많거나 SOR 비용 과다	ROI와 voxel 적용 상태 확인
dbscan이 큼	ROI/필터 후 포인트가 너무 많음	ROI, voxel, 주변 인원 확인
처리 10 Hz, 출력만 낮음	드론 검출 불안정	후보 점 수, cluster, gate 확인
PTP delta가 80 ms 초과 반복	두 센서 융합 시간차 문제	PTP와 센서 phase 상태 확인

현장에서는 두 debug 로그, 두 원본 토픽의 hz/bw, ip -s link 결과를 함께 보관한다. 이 네 가지가 있으면 센서 입력, 계산 병목, 네트워크 누락, 검출 실패를 서로 구분할 수 있다.